



Date ___ / ___ / ___

* LLMs vs. AI apps :-

LLM

- Statistical models trained on massive text corpora to predict & generate text
- Eg. GPT-4, Claude
- Transformer based with billions of params
- Understands language, generates text
- Takes sequence of tokens as input, O/Ps - probability distribution over next token
- No state, No memory

- AI Appⁿ - Systems built on top of LLMs. Combining the LLM with Memory layers (DBs, vector stores)
- Tools (APIs, Search engines)
 - Orchestration logic (When to call, what & in which order)
 - Business logic

LLM

Appⁿ

- Model
- No memory
- Stateless
- Generates text

System
Uses DBs / APIs
Stateful
Solve real world tasks

* Orchestration Framework :- (AI system becomes a many steps, many tools multiple agents.)

Workflow execution

Used to coordinate everything - How to chain steps together - which tool to call, when to call LLM, what context to pass, etc.

FWS - LangChain, LlamaIndex, CrewAI, LangGraph handle this Coordⁿ layer



w/o Orchestration fw :- Manually manage:

- Constructing prompts with right context
- Calling external AI
- Parsing LLM O/Ps & routing
- Retrying
- Managing convo history

Orchestratⁿ fws give you :-

- i> Chains - Connect multiple LLMs tools.
Output of Step A feeds into step B automatically.
 - ii> Agents - LLM decides at runtime, which & what tools to use in which order
ReAct loop
 - iii> Memory Management - Automatically summarize store & inject conversatⁿ history into prompt
 - iv> Tools / functⁿ Calling - Structured interface for LLM to invoke external tools (search, db) & receive results back.
- Observability - Tracing, logging & debugging every step of multi-step pipeline.



* Stateless Vs. Stateful Workflows :-

- | | |
|---|---|
| <ul style="list-style-type: none"> - Each request is completely independent. - No memory of previous interactions - fast & cheap per request - Simple to scale horizontally
Any server/server^s can handle any request. - Eg - You send a doc & get summarizatⁿ back.
It doesn't care what you sent before. - Doc processing, Classification transⁿ | <ul style="list-style-type: none"> Dependent Remembers all interactions State is persisted & retrieved when needed |
|---|---|

Types of State :-

- (i) Short term (Conversatⁿal) - Message history within a session
- (ii) Long term (user memory) - Persistent facts about user.
- (iii) Task State - Progress like (Step 3 out of 7)
- (iv) World State - External data that changes (User acc balance)

→ State is stored in :-

Date ___/___/___

In memory (fast, lost on restart)
DBs (SQL, NoSQL)
Vector DBs (Semantic searches or retrieval)
Redis / Cache (fast session rate)

Eg - Customer support agent that remembers your account, previous tickets, current conversation.

More powerful but harder to scale, debug & maintain

* Prompt-driven Systems :-
set of instructions to give guided output.
Dynamic & Reusable
Role based Prompts

- Zero Shot, few Shot, Chain of thought, Role prompting, Step by step reasoning.
- System prompt - sets persona, rules, tone, constraints.
Written once applied to every interaction.
- Output format specificⁿ - Telling model to return JSON, markdown or specific schema.

* Tool Augmented AI.

* RAG :-

Solves → Hallucination

Pipeline → Phase 1 - Indexing

Documents → Chunking → Embedding → Vector Store
 ↓
 large docs to smaller
Model ↓
 Chunks to dense vector that captures semantic meaning
 Vectors are stored in db (Pinecone, ChromaDB)

Phase 2 → Retrieval

User Query → Embed → Similarity → Top K Chunks
 Query Search Retrieved

Phase 3 Generation

System Prompt + Received Chunk + User Query
 → LLM → Answer.



* LangChain - is an Orchestration framework.
- Middle layer bet. raw LLM and your appⁿ.

Ecosystem - Collection of tools / frameworks used to build AI app^s, RAG systems, chatbots, AI agents

1) Lang chain Core - fundamental building blocks interfaces of LangChain.

↳ Chain, prompts, output parsers
↳ Runnable s
↳ Executable pipelines

Prompt Template - Dynamic prompts (structured way to create dynamic prompts by inserting vars to predefined template)
Chat Models - Abstract LLM interface
Output parsers - Structured outputs

Separate core logic from integrations.

2) Community - Includes integrations for VectorDBs, Doc. loaders, LLM providers
pdf / websites hugging face
↳ load Pdfs, using FAISS, integrate AI

3) Lang Serve - deploy LangChain app^s using APIs.
- Built using FastAPI.
- Productⁿ ready web services

Features :- API serving, streaming support, FastAPI integration, Playground UI.
↳ testing pipelines
Productⁿ backend.

Date ____ / ____ / ____

Product
observability

Identify
failures

4) LangSmith - Monitoring, Debugging & Evaluation platform for langchain, ^{Tracing app}
Compare O/Ps
- Helps in visualization.

5) LangGraph - Built on top of LangChain.

Simple chains are linear, LangGraph handles non-linearity.

It handles loops, retries, memory branching.

Eg AI research agent - Search web, analyze result, decide next action, repeat if needed.

Requires cycle, state management.

LCEL (LangChain Expression Language)

- Workflow Composition

prompt1 = PromptTemplate (

template =

input_var = ['topic']

invoke - Used to take input.



* Models in LangChain :- (Language Models & Embedding)

Chat Completion Embeddings

1) Chat - Conversational LLMs designed for msg based interact
- Use structured messages } Chatbots, tutors, assistants

2) Completion Models - Predict & complete text.
Eg Text gen, Story gen

3) Embedding Models - Num. vector representation of text.
- Capture semantic meaning, contextual similarity.

* Local vs. Hosted Models :-

↓
Runs directly on machine

↓
Runs on cloud servers.
Eg. GPT4, Claude, Gemini

Llama, Mistral, Gemma

Text based prompts with variables.

* Prompt Template - dynamically create prompts for LLMs, So instead of many writing fixed prompts it allows reusability
- Placeholder

Eg. Explain {topic} in simple words for.

System message - AI behaviour instⁿ
Human - User query.



Dynamic prompt injection - inserting runtime info dynamically

* Output parsers - raw LLM responses into structured, usable formats.

* Chaining - Pipeline

Simple, Sequential, Parallel, Runnable Parallel

→ LangChain Components :-

- 1) Models
- 2) Prompts
- 3) Chains
- 4) Memory
- 5) Indexes
- 6) Agents

for eg. Chain - flow of translation model

I/P → LLM → trans → LLM → summary

Parallel chain I/P → LLM1 → LLM2 → report - combine output.

Condⁿ Feedback I/P → process → good = Thankyou based - Email to customer support



4. Indexes - Connects your app" with external knowledge
- Doc loader, Text Splitter, Vector Store, Retrievers.
5. Memory - Conversat" Buffer Memory (Transcript of recent msgs)
" Window Memory (keeps last N interact"ns)
Summarizer Based Memory (summarizes chat segments)
Custom Memory (Key facts or user preferences)